

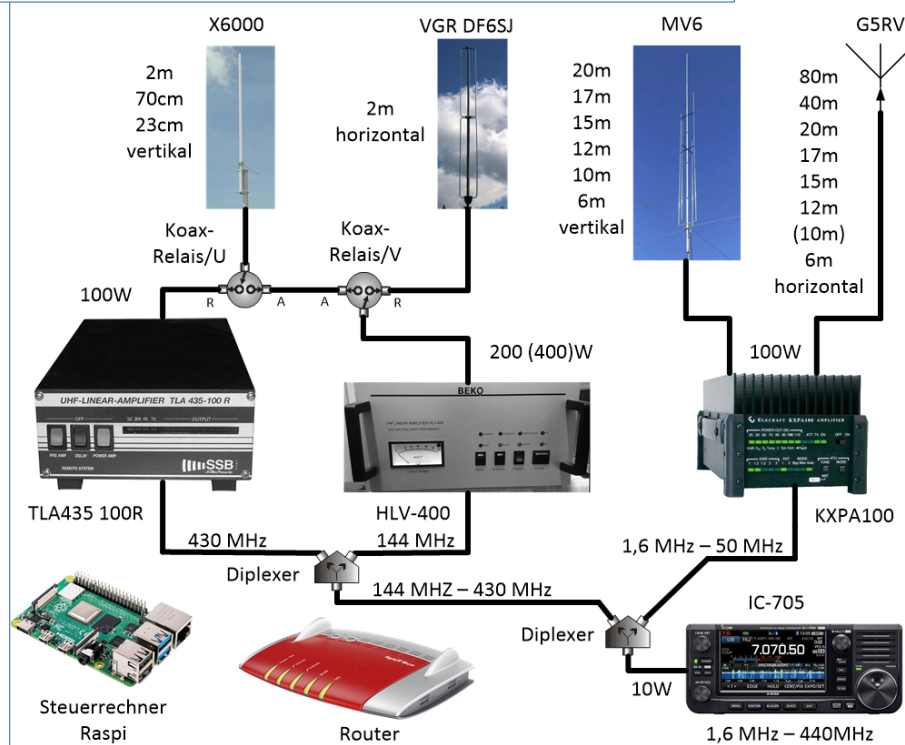
Beispiele CAT – Steuerung



- 1. ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- 2. YAESU FT-710 mit externem Tuner
 - 2.1 Steuerung über PC
 - 2.2 Steuerung über ESP32
- Verweise

Dieses Projekt zeigt die Steuerung von
Amateurfunkgeräten über die CAT-Schnittstelle
(Computer Aided Transceiver). Die Skripte wurden
auf einem Linux-Laptop entwickelt und für den
Einsatz auf einen Raspberry Pi sowie einen ESP32
portiert. Die Python-Skripte sind klein gehalten,
um die reine CAT-Kommunikation verständlich zu
zeigen.

1. Remote- Station mit IC-705



11.09.26 dk2jk

2/23

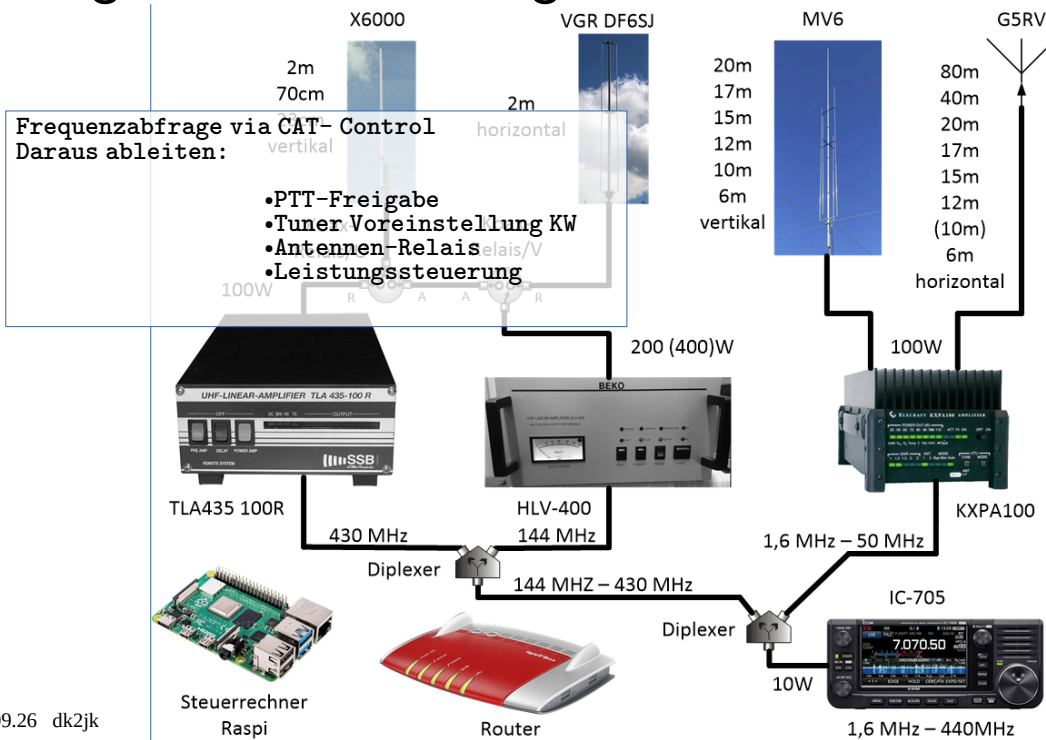
Im ersten Beispile wird eine Remote- Station mit einem IC-705 gezeigt; der Transceiver wurde erweitert um je eine Endstufe für 70cm, 2m und KW. Außerdem wurden mehrere fernsteuerbare Antennen vorgesehen.

Der Transceiver wird über die Remote- Software WFVIEW ferngesteuert.

Über WFVIEW ist jedoch die Zuschaltung individueller Endstufen und die Umschaltung der Antennen nicht möglich.

Hier kommt ein RaspberryPi ins Spiel, der über CAT-Steuerung die Frequenz abfragt und über ein Interface Steuersignale ausgibt.

Aufgabe der Steuerung



Die eingestellte Frequenz bestimmt fast alles!

Abhängig von der Frequenz wird der PTT-Eingang der jeweiligen Endstufen freigegeben.

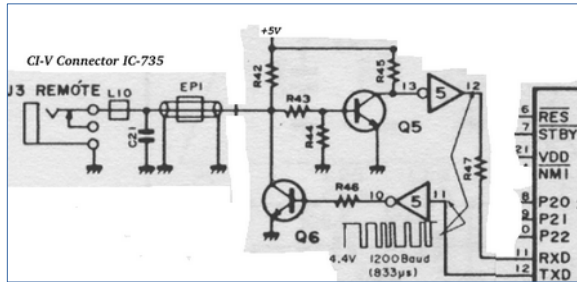
Der Tuner der Elecraft KXPA100 wird mit der aktuellen Frequenz voreingestellt, so dass der Tuner auch bei Empfang optimal arbeitet.

Weitere Steuerungs- Möglichkeiten, wie PA An- oder Aus- Schalten erfolgen über ein zusätzliches WEB- Interface; dieses ist aber hier kein Thema.

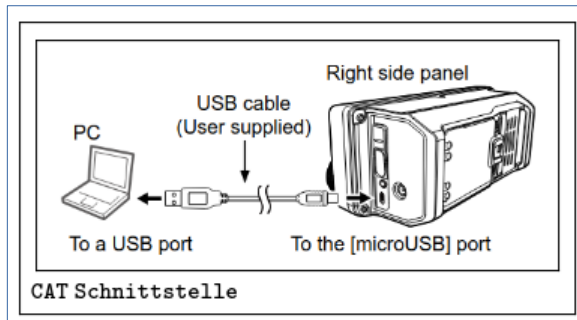
Beschäftigen wir uns nun mit der CAT- Steuerung des IC-705 ...

Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



CI-V „Computer Interface V (Fünf)
TTL -Level (Ruhezustand ca +5V,
Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung
(RXD und TXD) wird auf eine Leitung
kombiniert (Bus).



Beim IC-705 ist stattdessen eine
virtuelle serielle Schnittstelle
als **USB-Schnittstelle** vorhanden

11.09.26 dk2jk

4/23

So sieht die CAT- Hardware z.B. bei einem IC-735 aus.

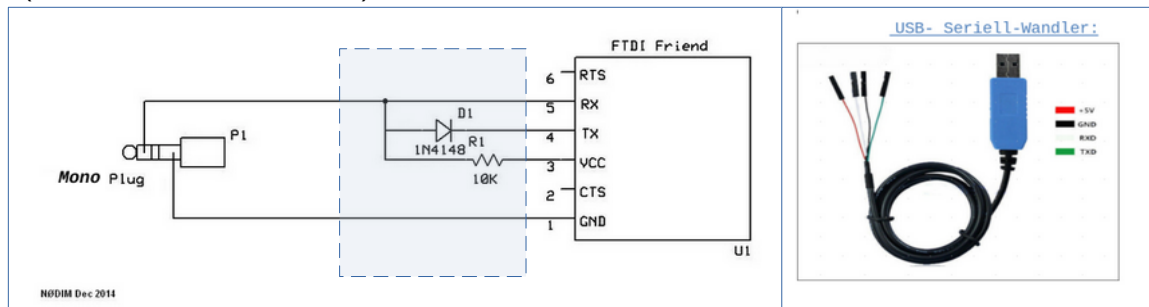
Grundsätzlich ist die CAT-Schnittstelle eine asynchrone serielle Schnittstelle mit Startbit , 8-Bit Daten, Stopbit. Die Baudrate wird im Transceiver eingestellt.

Bei ICOM heißt die Schnittstelle CI-V(fünf) . Ursprünglich ist es ein ‚Ein-Draht-Bus‘ mit TTL- Pegel. Es wird abwechselnd auf dem Bus gesendet oder empfangen; nur der Sender (TXD) zieht den Bus auf LOW. Das Schaltbild ist aus einem IC-735. (Anmerkung: da RXD und TXD die gleiche Signalleitung ist, hört der Sender sein TXD-Signal auf RXD mit.)

Beim IC-705 ist stattdessen eine virtuelle serielle Schnittstelle als USB-Schnittstelle vorhanden. Das ist bei den meisten modernen Transceivern der Fall. Die TTL-Schnittstelle ist dann oft noch extra vorhanden.

CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)



Adapter	---	CI-V Adapter
GND ,schwarz	---	G
TXD ,grün	---	rx
RXD ,weiß	---	tx
+5V	---	:

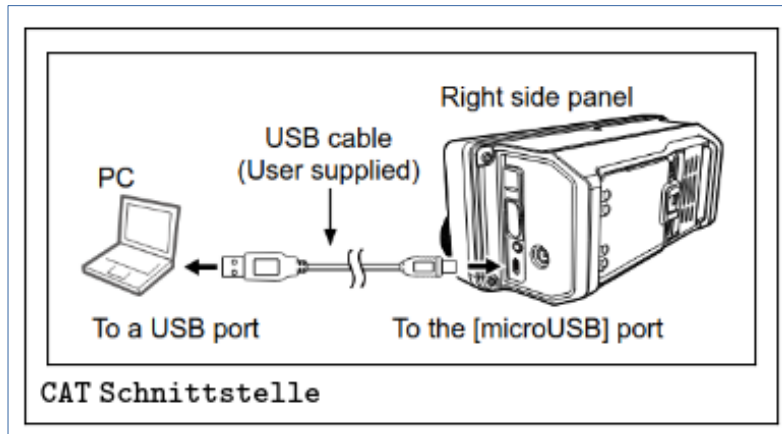
11.09.26 dk2jk

5/23

Für den Fall, dass eine USB-Schnittstelle benötigt wird, aber nur eine TTL- Schnittstelle vorhanden ist, so kann man mit einen USB- TTL- Adapter und ein wenig zusätzlicher Hardware einen USB-Adapter bauen.

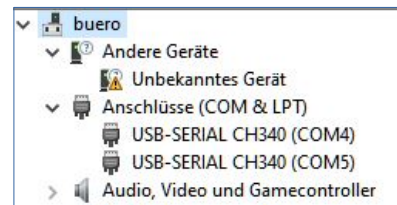
Hintergrund: Auf diese Weise konnte ich einen IC-706 über die USB-Schnittstelle eines Raspi steuern, da mir zu Beginn des Projekts kein IC-705 zur Verfügung stand.

USB als virtueller COM-Port



```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```

11.09.26 dk2jk



Zuerst stellt sich die Frage: „Wie heißt die Schnittstelle?“, mit der ich mich verbinden will.

Da der Raspi mit Linux läuft, schaue ich im Verzeichnis nach Geräten mit dem Namen „/dev/ttyA*“ (das ist der IC-705) und nach Geräten mit dem Namen „ttyU*“ (das ist die KXPA100).

Auf einem Windows-PC sucht man die Schnittstellen-Namen im Gerätemanager unter „Anschlüsse(COM)“

Die im folgenden vorgestellten Python-Programme für den Raspi laufen fast unverändert auf einem Linux- oder Windows-PC. Programmteile wurden am Laptop ausprobiert und später auf einen Raspi übertragen.

Ausschnitt IC-705-Command Table

◇ Command table

Cmd.	Sub cmd.	Data	Description
00		See p. 16.	Send the frequency data (transceive)
01		See p. 16.	Send the mode data (transceive)
02*1		See p. 16.	Read the band edge frequencies
03*1		See p. 16.	<u>Read the operating frequency</u>
04*1		See p. 16.	Read the operating mode
05*2		See p. 16.	Set the operating frequency
06*2		See p. 16.	Set the operating mode
07			Select the VFO mode

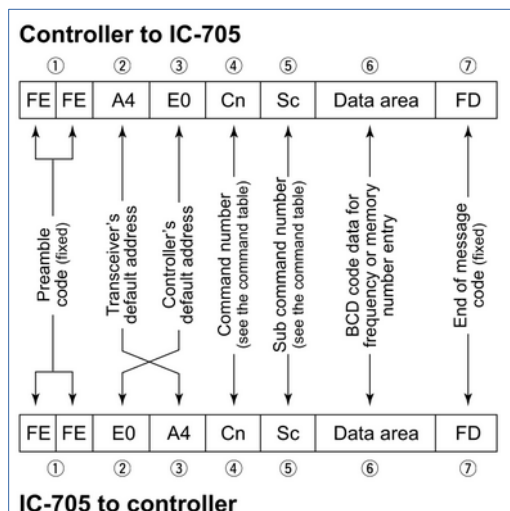
11.09.26 akzjk

In der Kommandotabelle des IC-705 suche ich nach dem Kommando für die Frequenzabfrage; dieses ist ,0x03' „read the operating frequency“.

Weitere Kommando für unsere Anwendung benötige ich zunächst nicht.

Für die Programmierung muss dieses Kommando in den erforderlichen Datenrahmen eingebaut werden.

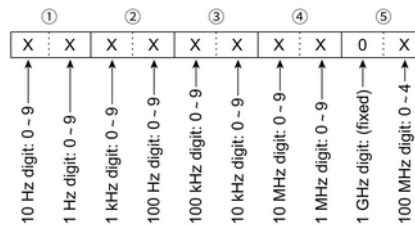
CI-V Command Format ICOM-705



◇ Command formats

• Operating frequency

Command: 00, 03, 05, 1C 03



11.09.26 dk2jk

Die CI-V- Kommandos werden in einem Datenrahmen codiert.

Bei ICOM werden die Daten hexadezimal dargestellt; ein Hex- Buchstaben-Paar entspricht einem Byte.

Die mit dem Hexwert verbundenen Werte als Zahl sind unwichtig; wir betrachten nur die HEX-Codes.

Als nächstes wird die Codierung am Beispiel der Frequenzabfrage erläutert...

Frequenzabfrage Datenformat

```
#ICOM Kommando zum Frequenz lesen:
# FE FE A4 E0 03 FD
# FE FE                <- kopf
# ..... A4            <- IC-705 CAT Adresse
# ..... E0            <- default ??
# ..... 03            <- Kommando Frequenz lesen
# ..... FD            <- Ende Zeichen
```

Interpretation der Antwort:

```
Antwort      = FE FE E0 A4 03 00 50 43 28 00 FD
Nutzdaten    = 00 50 43 28 00
Nutzdaten    = 00 28 43 50 00 (gedreht)
Roh-Ergebnis = "0028435000" = text
Frequenz     = 28.435 MHz = float(text)/1e6
```

Das Datenformat CI-V besteht aus:

Präambel	FE FE	
CAT-Adresse	A4	(für IC-705)
Controller Adresse	E0	(ich)
Kommando	03	(Frequenz lesen)
Ende-Zeichen	FD	

Die Antwort besteht aus den Daten des Kommandos, wobei Sender und Empfänger vertauscht sind ; dann folgen 5 Bytes Nutzdaten; Abgeschlossen wird das Protokoll mit dem Hexwert 0xFD. Wenn man die Nutzdaten von hinten liest, so kann man an den HEX-Bytes schon die Frequenz lesen; das Ergebnis muss noch durch 1.000.000 geteilt werden, so erhält man die Frequenz in MHz.

Frequenzabfrage ICOM Python Programm

```
[ ic705_cat_mit kommentar.py ]
1
2 from serial import Serial      # modul serial laden
3
4                                     # Kommentar mit Beispieldaten...
5 def read_frequenz_IC_705():     # Frequenz lesen ...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD]
7                                     # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )      # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]           #
12    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 50 12 07 00
13    text = nutzdaten_rueckwaerts.hex()  # text = "0007125000"
14    mhz = float(text)/1000_000          # mhz = float("0007125000")/1e6
15    com.close()                       # schnittstelle schliessen
16    return mhz                        # ergebnis = 7.125
17
18 if __name__ == '__main__':
19     frequenz = read_frequenz_IC_705() # frequenz lesen
20     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz

Shell >
>>> %Run 'ic705_cat_mit kommentar.py'
f=7.125 MHz
```

11.09.26

Mit diesem Python Skript wird die Frequenz gelesen.

Zeile 19: Funktion ,read_frequenz_IC705 wird aufgerufen, welche die Frequenz liefert.

Zeile 6: das Kommando wird zusammengesetzt (eine Liste mit Werten, hier in hex eingegeben.)

Zeile 8: Schnittstelle wird eröffnet; der Name der Schnittstelle wurde vorher im Gerätemanager abgelesen.

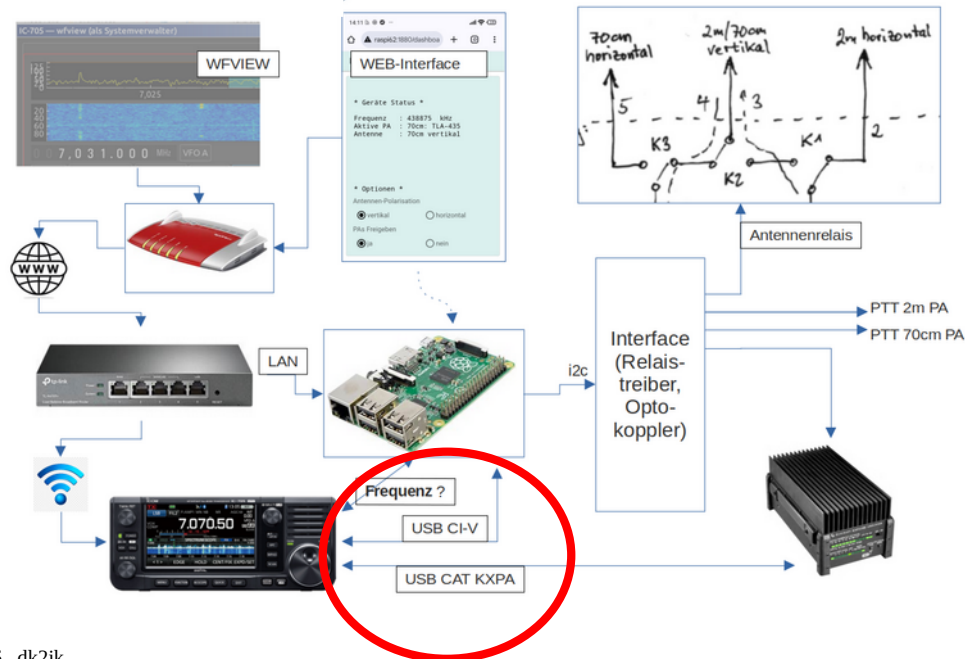
Zeile 9: Daten werden als Bytes geschrieben.

Zeile 10: Antwort wird gelesen (hier andere Frequenz als im vorherigen Beispiel)

Zeile 11 ...13: Nutzdaten werden herausgefiltert und formatiert

Zeile 14: Ergebnis in MHz

Wie hängt alles zusammen?



11.09.26 dk2jk

11/23

Hier ist nochmals die ganze Hardware der Remote-Station schematisch abgebildet.

Die Hauptaufgabe des Raspi ist es, die Frequenz über die CAT-Schnittstelle zu lesen.

Abhängig von der eingestellten Frequenz werden über ein I2C-Interface die passenden Antennen-Relais geschaltet. Außerdem wird die PTT der zugehörigen PA freigegeben. Der Tuner in der KXPA100 erhält ebenfalls die Frequenz-Info, damit der Tuner auch bei Empfang schon richtig steht.

Über ein zusätzlichen WEB-Interface ("Dashboard") werden die PAs ein- oder aus-geschaltet. Bei UKW kann von vertikaler auf horizontale Polarisierung umgeschaltet werden.

Das WEB-Interface wird hier nicht näher betrachtet (Stichworte hierzu : NodeRED, MQTT)

CAT-Control KXPA100

Befehlsformat

KXPA100-Befehle beginnen in der Regel mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon abgeschlossen
(;).

**Der Tuner wird auf die Frequenz geschaltet
noch bevor er überhaupt HF sieht.**

über CAT-Schnittstelle
(einfaches ASCII-Format)

Befehl: ^MT7032;
== Memory Tune 7032 kHz

11.09.26 dk2jk

Hier eine kurze Info , wie ein CAT- Befehl bei der
KXPA100 aussieht.

Nebenbei:

Die CAT-Befehle des K2, K3 oder KX3 sehen ähnlich aus
(Ohne (^)).

Da die Befehle in ASCII codiert sind, kann man sie
ohne Umwandlung als Text lesen.

Bei Kennwood und YAESU sind die Befehle ebenfalls in
ASCII codiert.

2. Tune-Button für ext. Tuner am FT-710

Ein externer Tuner soll zum Abstimmen mit einem 10W- Träger angesteuert werden.

Möglichkeiten:

1. Manuell:
Mode=AM (ca. 25W) --> PTT am Mikrofon drücken -->Tuner Arbeitet --> zurück auf alten ,Mode'
2. Verwendung der Buchse „REM/ALC“
Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am Signal ALC; - 4.9V --> 10W. (TUN/LIN PORT SELECT auf ,Linear')
3. CAT-Schnittstelle:
eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet.

11.09.26 dk2jk

In einem zweiten Beispiel wird ein externer Tuner zum Abstimmen mit einem 10W Träger angesteuert.

Es gibt 3 Möglichkeiten um einen Träger zum Abstimmen des Tuners zu senden.

1. Manuell: das ist mir zu umständlich

2. ALC: Funktioniert, man benötigt einen 3,5 mm Mono Stecker, einen Taster und eine negative Spannung, z.B. ein 9V Batterie plus Poti.

3. CAT: diese Schnittstelle will ich benutzen !!

Python Programm Frequenzabfrage YAESU

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

```
cat_pc_basic.py <
12
13 port = '/dev/ttyUSB0'      # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
20     time.sleep(0.2)           # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()    # bytes in ascii umwandeln
23

Shell <
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

11.09.26 dk2jk

Bei YAESU sieht ein CAT- Befehl wesentlich einfacher aus als bei ICOM, da die Befehle ASCII-codiert sind.

Dazu habe ich ein Python-Skript geschrieben mit der Funktion `cat()` mit dem Argument `,cmd'` == Befehl als Übergabewert in der Klammer.

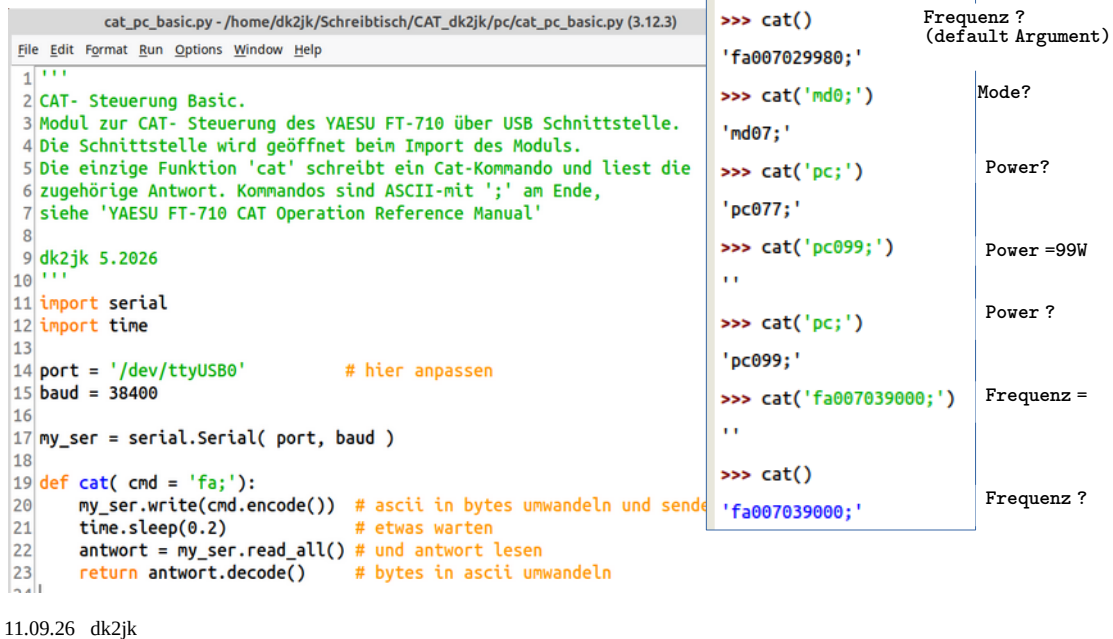
Im Beispiel wird die Frequenz mit dem Befehl `,fa;'` abgefragt.

Übrigens: CAT-Befehle dürfen grundsätzlich groß oder klein geschrieben werden.

Diese Funktion würde auch bei Elecraft K3, KX3 oder Kenwood funktionieren.

Kenwood habe ich nicht ausprobiert.

Experimente live



The screenshot shows the Thonny Python IDE with a file named `cat_pc_basic.py` open. The script is a Python program that controls a CAT-710 radio via a USB interface. It includes comments in German explaining its purpose and usage. The script sets the serial port to `/dev/ttyUSB0` and the baud rate to `38400`. It defines a function `cat(cmd)` that sends a command to the radio and reads the response. The script is executed, and the output is shown in the Shell window. The output shows the command `cat()` being executed, which sends the command `'fa007029980;'` to the radio. The response is `'md07;'`, which is interpreted as `Mode?`. The command `cat('pc;')` is then executed, sending `'pc077;'` to the radio, which responds with `'pc099;'`, interpreted as `Power = 99W`. The command `cat('fa007039000;')` is then executed, sending `'fa007039000;'` to the radio, which responds with `'fa007039000;'`, interpreted as `Frequenz =`.

```
cat_pc_basic.py - /home/dk2jk/Schreibtisch/CAT_dk2jk/pc/cat_pc_basic.py (3.12.3)
File Edit Format Run Options Window Help
1 '''
2 CAT- Steuerung Basic.
3 Modul zur CAT- Steuerung des YAESU FT-710 über USB Schnittstelle.
4 Die Schnittstelle wird geöffnet beim Import des Moduls.
5 Die einzige Funktion 'cat' schreibt ein Cat-Kommando und liest die
6 zugehörige Antwort. Kommandos sind ASCII-mit ';' am Ende,
7 siehe 'YAESU FT-710 CAT Operation Reference Manual'
8
9 dk2jk 5.2026
10 '''
11 import serial
12 import time
13
14 port = '/dev/ttyUSB0'          # hier anpassen
15 baud = 38400
16
17 my_ser = serial.Serial( port, baud )
18
19 def cat( cmd = 'fa;'):
20     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und sende
21     time.sleep(0.2)           # etwas warten
22     antwort = my_ser.read_all() # und antwort lesen
23     return antwort.decode()    # bytes in ascii umwandeln
24
11.09.26 dk2jk

Shell
>>> cat()
'fa007029980;'
>>> cat('md0;')
'md07;'
>>> cat('pc;')
'pc077;'
>>> cat('pc099;')
'pc099;'
>>> cat('fa007039000;')
'fa007039000;'
>>> cat()
'fa007039000;'

Frequenz ?
(default Argument)
Mode?
Power?
Power = 99W
Power ?
Frequenz =
Frequenz ?
```

Datei: /home/dk2jk/Schreibtisch/CAT_dk2jk/pc/
cat_pc_basic.py

Bei dieser Live-Vorführung wird das Programm THONNY verwendet, eine Python IDE mit Dateimanager, Editor und Shell. Thonny ist besonders attraktiv, da es nur wenige Einstellungen gibt und außerdem in Verbindung mit allen Micropython- Prozessoren läuft.

Rückblende zum IC-705 Projekt:

Mit Thonny IDE kann man auch fernbedient programmieren (Stichwort SSH). Thonny läuft dann auf meinem Laptop, die Python Skripts laufen aber auf dem fernen Raspi.

Zu Thonny gibt es einen Link auf der letzten Seite.

2.1. Tune Programm am PC

Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write ( beep )  
    close() # Schnittstelle schliessen  
    return
```

Komplettes Programm im
Anhang

11.09.26 dk2jk

Hier sind alle Befehle zusammengefasst für einen Tune-Button mit den Funktionen `get()` und `write()`. Weiterhin sind noch einige Feinheiten ergänzt. So wird die Messanzeige (,meter') während der Tuner-Abstimmung auf ,SWR- Anzeige' eingestellt, so dass der Abstimm- Vorgang am SWR beobachtet werden kann. Außerdem wird die Lautstärke (,AF-Gain') kurzfristig auf Null gestellt.

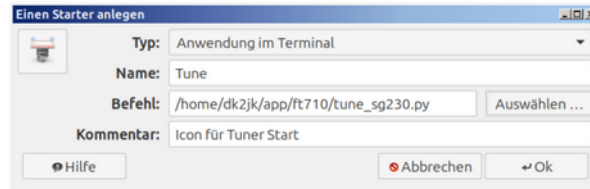
Im kompletten Programm sind noch einige Fehlermöglichkeiten abgefangen (z.b. Schnittstelle nicht vorhanden).

Zur Ansicht oder zum Testen des kompletten Programms steht ein Download zur Verfügung; dazu befindet sich ein Link auf der letzten Seite.

Tune Icon Link (Linux)

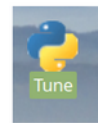
Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon = python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



11.09.26 dk2jk

Das Tune-Programm wird über einen Knopfdruck (Icon) aufgerufen.

Bei Linux wird ein Starter-Icon angelegt. Im wesentlichen wird hier nach „Exec=“ das eingetragen, was man auch im Terminal zum Start des Python-Scripts eintippen würde:

z.B.:

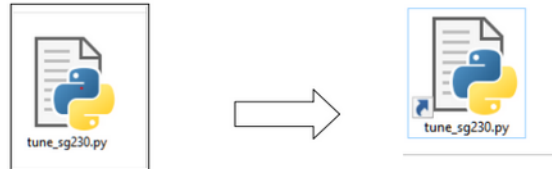
Exec=python3 /home/dk2jk/app/FT-710/tune_sg230.py

Das Icon kann man noch mit einem individuellen Symbol gestalten.

Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:



Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

11.09.26 dk2jk

Bei Windows wird im Dateimanager aus dem Python-Skript-Icon ein Verknüpfungs-Icon gemacht. Diese wird dann auf den Desktop verschoben.
Das Python-Skript muss die Eigenschaft „ausführbar“ haben.

4-pol. Kabel (nur für Dokumentation)

FT-710
CAT-3 Connector
8-Pol-'Mini-Din'

Cat-3.1
Cat-3.2
Cat-3.3
Cat-3.4
Cat-3.5
Cat-3.6
Cat-3.7
Cat-3.8
Cat-3.9

13_8V
GND
TXD_710
RXD_710

5 Volt vom USB des ESP-32- Moduls
5V_USB
Convert to PCB : no

U1
GPIO5
GPIO6
GPIO7
GPIO8
GPIO9
GPIO10
GPIO20
GPIO21
5V_ESP
GND
3V3
3.3 Volt Regler befindet sich auf dem ESP-32 Modul

ESP32-C3

H2 Taster
H3 Taster

GND

13_8V
VIN
GND
OUT
LM7805T
D1
D2
D3
1N4001
C1
C2
22uF
100nF
5V_ESP

TUNER/LINEAR Jack

⑤ RX D
④ TX D
③ GND
②
① +13V OUT

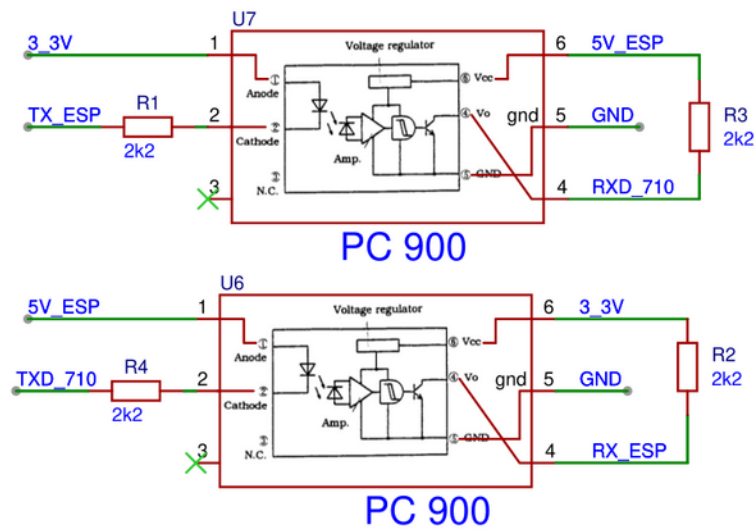
(as viewed from rear panel)

In diesem Beispiel löst ein Taster die gleiche Befehlsfolge aus, wie sie beim FT-710 beschrieben wurde; hier jedoch nicht vom Laptop aus sondern über einen realen Taster am Mikroprozessor.

Da hier der Prozessor mit 3.3 Volt arbeitet, muss für RXD des Prozessors ein Spannungsteiler 5V/ 3,3V vorgeschaltet werden. Der Sender TXD des Prozessors ist ok, da 3,3V als High-Pegel am RXD Eingang des Transceivers als TTL-HIGH akzeptiert wird.

Python läuft als Micro-Python in vielen 32-bit-Prozessoren. Dazu wird in den Prozessor zunächst ein Micropython- Image geladen. Die Kommunikation geht nur über die serielle Schnittstelle (siehe Anhang „Micropython“)

Variante mit Optokoppler



11.09.26 dk2jk

Der Vollständigkeit halber wird hier noch eine Variante des 3,3V/ 5V Pegelwandlers mit Optokoppler gezeigt. Der der Spannungsteiler ist einfacher.

MicroPython Programm ESP32

```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write( beep )
    close()
    return
```

```
main.py *
#main.py
import esp32_tune
```

```
esp32_tune.py
while True:
    x= taste() # taste lesen , low aktiv
    led(x) # led an solange taste gedrueckt ist
    if x ==0: # taste gedrueckt ?
        tune_function()
```

11.09.26 dk2jk

Hier sind nochmals alle Befehle zusammengefasst für einen Tune- Button. Die Funktion selbst ist identisch mit der PC-Version.

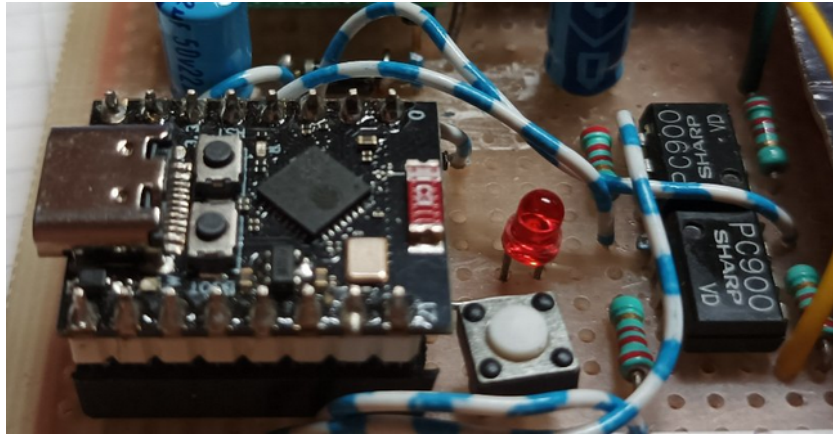
Nur die Funktionen `get()` und `write()` sehen für Micropython etwas anders aus.

Der Tune- Button (Taste) wird in einer Schleife dauernd abgefragt. Eine optische Kontrolle ist über LED vorgesehen. Wenn die Taste gedrückt wird, so wird die `Tune_funktion` gestartet.

Damit das Programm beim Einschalten automatisch startet, muß man bei Micropython in der Datei `,main.py'` das Tune Programm aufrufen ,siehe Ausschnitt oben rechts. Auf diese Weise wird in Micropython ein Autostart gemacht.

Das Programm steht zum Download zur Verfügung.

Bild „Tune Button mit ESP32“



11.09.26 dk2jk

Verweise

Diese Präsentation	https://dk2jk.darc.de/archiv/CAT_dk2jk/Beispiele-CAT-Steuerung.pdf
Python-Skripts	https://dk2jk.darc.de/archiv/CAT_dk2jk/CAT--Control-main.zip
Alle Unterlagen auf Github	https://github.com/dk2jk/CAT--Control
Python Download für verschiedene Betriebssysteme	https://www.python.org/downloads/
Thonny Python-Entwicklungs-Oberfläche (IDE)	https://thonny.org/
MicroPython Download für Microcontroller	https://micropython.org/download/
ICOM CI-V REFERENCE GUIDE (English)	https://www.icomjapan.com/support/manual/3064/
YAESU CAT Reference	https://www.radiomanual.info/schemi/YAESU_HF/FT-710_CAT_user.pdf
Elecraft KXPA100 Amplifier Command Reference	https://ftp.elecraft.com/KXPA